

Functions

What are Functions?

Think of **functions** as blocks of code that complete a specific task on a set of inputs.

```
x <- 1  
y <- 2  
z <- sum(x, y)
```

In the example above, when you **called** (used or invoked) the `sum()` function on `x` and `y`, it produced the sum of 1 and 2.

Pro Tip:

You do NOT need to memorize every available R function! Although you will start remembering the common functions you use, computer programmers look up documentation for functions all the time.

Demo

Function Documentation

In this demo, we will:

Look up function documentation in RStudio.

Defining Functions

In addition to calling functions build into R, you can **define** your own functions.

```
convert_in_cm <- function(measurement, unit) {  
  if (unit == "in"){  
    conversion <- measurement * 2.54  
  } else if (unit == "cm"){  
    conversion <- measurement / 2.54  
  }  
  return(conversion)  
}  
  
result <- convert_in_cm(measurement=5, unit="in")
```

Let's break this down...

Functions contain **arguments**.

Functions are stored in objects.

Functions **return** the value within the **return()** function

```
convert_in_cm <- function(measurement, unit) {  
  if (unit == "in"){  
    conversion <- measurement * 2.54  
  } else if (unit == "cm"){  
    conversion <- measurement / 2.54  
  }  
  return(conversion)  
}  
  
result <- convert_in_cm(measurement=5, unit="in")
```

After functions are **defined**, they can be **called**.

Functions contain
arguments.

Functions are
stored in objects.

Functions **return** the
value within the
return() function

```
convert_in_cm <- function(measurement, unit) {  
  if (unit == "in"){  
    conversion <- measurement * 2.54  
  } else if (unit == "cm"){  
    conversion <- measurement / 2.54  
  }  
  return(conversion)  
}  
  
result <- convert_in_cm(measurement=5, unit="in")
```

After functions are **defined**,
they can be **called**.

Functions contain
arguments.

Functions are
stored in objects.

Functions **return** the
value within the
return() function

```
convert_in_cm <- function(measurement, unit) {  
  if (unit == "in"){  
    conversion <- measurement * 2.54  
  } else if (unit == "cm"){  
    conversion <- measurement / 2.54  
  }  
  return(conversion)  
}  
  
result <- convert_in_cm(measurement=5, unit="in")
```

After functions are **defined**,
they can be **called**.

Functions contain **arguments**.

Functions are stored in objects.

Functions **return** the value within the **return()** function

```
convert_in_cm <- function(measurement, unit) {  
  if (unit == "in"){  
    conversion <- measurement * 2.54  
  } else if (unit == "cm"){  
    conversion <- measurement / 2.54  
  }  
  return(conversion)  
}  
  
result <- convert_in_cm(measurement=5, unit="in")
```

After functions are **defined**, they can be **called**.

Functions contain
arguments.

Functions are
stored in objects.

Functions **return** the
value within the
return() function

```
convert_in_cm <- function(measurement, unit) {  
  if (unit == "in"){  
    conversion <- measurement * 2.54  
  } else if (unit == "cm"){  
    conversion <- measurement / 2.54  
  }  
  return(conversion)  
}  
  
result <- convert_in_cm(measurement=5, unit="in")
```

After functions are **defined**,
they can be **called**.

Functions contain **arguments**.

Functions are stored in objects.

Functions **return** the value within the **return()** function

```
convert_in_cm <- function(measurement, unit) {  
  if (unit == "in"){  
    conversion <- measurement * 2.54  
  } else if (unit == "cm"){  
    conversion <- measurement / 2.54  
  }  
  return(conversion)  
}  
  
result <- convert_in_cm(measurement=5, unit="in")
```

After functions are **defined**, they can be **called**.

Knowledge Check

Calling and Defining
Functions

```
1  convert_fahrenheit_celcius <- function(temperature, unit) {  
2    if (unit == "F"){  
3      conversion <- (temperature - 32) * (5 / 9)  
4    } else if (unit == "C"){  
5      conversion <- (9 / 5) * temperature + 32  
6    }  
7    return(conversion)  
9  }  
10  
11 convert_fahrenheit_celcius(temperature=-14, unit="F")
```

Answer the following questions:

- In which line number is the function called?
- What is the name of the object that contains the function?
- What arguments does the function contain?
- In which line number is the function returning a value?

Knowledge Check

Calling and Defining Functions

```
1  convert_fahrenheit_celcius <- function(temperature, unit) {  
2    if (unit == "F"){  
3      conversion <- (temperature - 32) * (5 / 9)  
4    } else if (unit == "C"){  
5      conversion <- (9 / 5) * temperature + 32  
6    }  
7    return(conversion)  
9  }  
10  
11 convert_fahrenheit_celcius(temperature=-14, unit="F")
```

Answer the following questions:

- In which line number is the function called? 11
- What is the name of the object that contains the function?
convert_fahrenheit_celsius
- What arguments does the function contain? temperature, unit
- In which line number is the function returning a value? 7

Demo

Installing Packages

In this demo, we will:

Install the tidyverse



University of California
San Francisco